

What Are Microservices?

Small, independent services that do **one job** and talk to other services using APIs.

Key points:

- Each service has its **own codebase**
- Each service can be **built, deployed, and scaled independently**
- Services communicate using **REST, gRPC, Kafka** etc.

Why Microservices? (Interview-ready reasons)

- Faster development
- Independent deployments
- Easy scaling
- Better fault isolation (if one fails, system survives)
- Can use different tech stacks

How Services Talk to Each Other

Sync communication

- REST API
- gRPC

Async communication (preferred)

- Kafka
- RabbitMQ
- SQS

Async = less dependency, better performance.

Service Discovery (Very Important)

Used to find locations (IP/port) of other services.

Tools:

- Eureka
- Consul
- Zookeeper
- Kubernetes Service Discovery

Without discovery → services can't find each other in dynamic environments.

API Gateway

Acts like a **single entry point** for all microservices.

Functions:

- Routing
- Authentication
- Throttling
- Logging
- Rate limiting
- Hiding internal services

Examples:

- Spring Cloud Gateway
- Kong
- NGINX
- Apigee

Load Balancing

Used to distribute traffic across service instances.

Options:

- Ribbon (deprecated)
- Spring Cloud LoadBalancer
- NGINX
- AWS ALB / ELB

- Envoy

Centralized Configuration (Config Server)

Store external configs in one place.

Example:

- Spring Cloud Config Server
- Git-based config
- Vault for secrets

Purpose:

- No need to redeploy app for config change
- Version control for configs

Circuit Breakers (Very Common Question)

Prevents cascading failure if one service is slow or down.

Tools:

- Resilience4j (most used)
- Netflix Hystrix (Retired)

Patterns:

- Retry
- Fallback
- Timeout
- Bulkhead
- RateLimiter

Distributed Logging

Microservices = many containers → logs scattered.

Solutions:

- ELK Stack (Elastic + Logstash + Kibana)

- EFK (Elastic + Fluentd + Kibana)
- Grafana Loki
- CloudWatch / Stackdriver

Distributed Tracing

Helps track requests across multiple services.

IDs used:

- TraceId
- SpanId

Tools:

- Zipkin
- Jaeger
- OpenTelemetry

Docker + Kubernetes

Microservices run inside **Docker containers**.

Kubernetes handles:

- Scaling
- Self-healing
- Deployment
- Rolling updates

Database Per Service

Each service should have **its own database**.

Avoid:



sharing

DB

 cross-service joins

Use patterns:

- API composition
- CQRS
- Event sourcing

Event-Driven Architecture (Very Important)

Replace REST calls with **events**.

Events examples:

- OrderCreated
- PaymentCompleted
- UserRegistered

Benefits:

- Loose coupling
- Better performance
- Async processing

SAGA Pattern (Distributed Transactions)

Used when multiple services must maintain consistency.

Two types:

- **Orchestration Saga** → one central controller
- **Choreography Saga** → services react to events (no controller)

Example:

Order Service → Payment Service → Shipping Service

Outbox Pattern

Ensures reliable communication between DB and Kafka.

Prevents:

- Lost events
- Duplicate events

API Versioning

Required in microservices since many teams deploy independently.

Version using:

- URL (v1, v2)
- Headers
- Query params

Authentication & Authorization

Common tools:

- OAuth2
- JWT
- Keycloak
- Okta
- AWS Cognito

API Gateway usually handles auth.

Fault Tolerance Patterns (Quick Summary)

- Retry
- Circuit breaker
- Rate limit
- Timeout
- Bulkhead (isolate failures)
- Fail fast

Deployment Patterns

- Blue-Green deployment
- Canary deployment

- Rolling updates
- A/B testing

Most Asked Microservice Interview Questions

1. How do services communicate?
2. How do you make microservices fault-tolerant?
3. What is Saga?
4. Why do we need API Gateway?
5. How do you handle distributed logging?
6. How do you handle distributed tracing?
7. Why database-per-service?
8. Difference between synchronous vs asynchronous communication.
9. How do you deploy microservices?
10. What problems do microservices solve?